

Este proyecto es sumamente vistoso, 100% interactivo en el emulador, y unifica el desarrollo *wearable* con el consumo de servicios en la nube de forma ágil.

Tutorial: Sistema de Control IoT y Clima en Wear OS

Este proyecto práctico enseña a transformar un *smartwatch* en un nodo de control remoto para sistemas embebidos e Internet de las Cosas (IoT), gestionando peticiones HTTP asíncronas y adaptando componentes de control (como interruptores y selectores) al espacio restringido de un *wearable*.

1. Fundamentos Teóricos y Conceptos del Proyecto

Al desarrollar una aplicación de control IoT para pantallas circulares, se deben dominar tres conceptos de arquitectura y diseño:

- **Peticiones Asíncronas en Redes Reducidas (FutureBuilder):** El reloj no debe almacenar bases de datos pesadas. Su función es ser un cliente ligero que consulta el estado de los servidores de forma asíncrona. Utilizaremos el estado del clima de una API real para simular sensores ambientales.
- **Patrón de Diseño de Desplazamiento Circular (ScalingLazyColumn):** En Wear OS, las listas lineales comunes se ven mal porque las opciones superiores e inferiores se cortan en las pantallas redondas. Aunque en Flutter básico usamos `ListView`, la buena práctica en relojes es diseñar con contenedores centrados que escalen su tamaño dinámicamente conforme se deslizan.
- **Micro-Interacciones:** Un reloj no es para escribir texto. Toda la interacción debe resolverse con toques únicos (*Single Taps*), interruptores (`Switch`) o selectores de incremento/decremento (+ y -).

2. Configuración del Entorno (Plataforma Nativa)

Dado que el emulador del reloj necesitará conectarse a Internet para jalar el estado del clima y enviar comandos IoT simulados, debemos asegurar el permiso de red.

En Android (`android/app/src/main/AndroidManifest.xml`)

Asegúrate de que la línea de permiso de Internet esté declarada justo arriba de la etiqueta `<application>` (junto a la característica de reloj que configuramos en la sesión anterior):

```
<uses-permission android:name="android.permission.INTERNET" />
<uses-feature android:name="android.hardware.type.watch" />
```

Estructura de Carpetas del Proyecto IoT

Dentro de nuestro proyecto dedicado para el reloj, crearemos una estructura limpia dividida por pantallas y servicios simulados:

```
mi_proyecto_wearable/  
├── pubspec.yaml  
└── lib/  
    ├── main.dart  
    ├── app.dart  
    └── screens/  
        └── watch_iot_screen.dart    <-- Panel de control principal IoT
```

Dependencias del `pubspec.yaml`

Para esta práctica utilizaremos el paquete `wear_plus` que ya tenemos configurado, y añadiremos el cliente HTTP estándar de Dart:

```
dependencies:  
  flutter:  
    sdk: flutter  
  wear_plus: ^2.1.1  
  http: ^1.2.0      # Para consumir datos reales de internet
```

3. Código Fuente Completo y Comentado

Archivo: `lib/screens/watch_iot_screen.dart`

Sustituye o crea este archivo. El código simula el control de dos lámparas inteligentes del hogar y consume datos reales de temperatura.

```
import 'package:flutter/material.dart';  
import 'package:wear_plus/wear_plus.dart';  
import 'dart:convert';  
import 'package:http/http.dart' as http;  
  
class WatchIotScreen extends StatefulWidget {  
  const WatchIotScreen({super.key});  
  
  @override  
  State<WatchIotScreen> createState() => _WatchIotScreenState();  
}  
  
class _WatchIotScreenState extends State<WatchIotScreen> {  
  // Estados locales de los actuadores IoT de la casa  
  bool _livingRoomLight = false;  
  bool _kitchenLight = false;  
  int _targetTemperature = 22; // Control del termostato
```

```
// 1. Simulación de consumo de API de Clima (HTTP Asíncrono)
Future<double> _fetchCurrentWeather() async {
  try {
    // Consumimos una API pública de geocodificación y clima (CDMX de ejemplo)
    final url = Uri.parse('https://api.open-meteo.com/v1/forecast?
latitude=19.4326&longitude=-99.1332&current_weather=true');
    final response = await http.get(url);

    if (response.statusCode == 200) {
      final data = jsonDecode(response.body);
      return data['current_weather']['temperature'];
    }
  } catch (e) {
    debugPrint("Error de red: $e");
  }
  return 24.5; // Valor de respaldo (fallback) si el laboratorio no tiene
internet
}

@override
Widget build(BuildContext context) {
  return WatchShape(
    builder: (context, shape, child) {
      return AmbientMode(
        builder: (context, mode, child) {
          final bool isAmbient = mode == WearMode.ambient;
          final bool isRound = shape == WearShape.round;

          return Scaffold(
            backgroundColor: Colors.black,
            body: Center(
              // Implementamos un scroll básico para que los alumnos puedan
              // deslizar verticalmente el pequeño panel de control en el
              emulador

              child: SingleChildScrollView(
                padding: EdgeInsets.symmetric(
                  horizontal: isRound ? 16.0 : 8.0,
                  vertical: isRound ? 24.0 : 12.0,
                ),
                child: Column(
                  mainAxisAlignment: MainAxisAlignment.min,
                  children: [
                    // --- SECCIÓN 1: TELEMETRÍA (API DE CLIMA) ---
                    FutureBuilder<double>(
                      future: _fetchCurrentWeather(),
                      builder: (context, snapshot) {
                        String tempText = snapshot.hasData ? "${snapshot.data}
°C" : "...";

                        return Row(
                          mainAxisAlignment: MainAxisAlignment.center,
                          children: [
                            Icon(Icons.wb_sunny, color: isAmbient ? Colors.grey
: Colors.amber, size: 14),
                            const SizedBox(width: 4),
```

```

        Text(
          "Ext: $tempText",
          style: const TextStyle(fontSize: 11, fontFamily:
'monospace'),
        ),
      ],
    );
  },
),
const SizedBox(height: 8),

// --- SECCIÓN 2: ACTUADOR LÁMPARA SALA ---
_buildIotRow(
  icon: Icons.lightbulb,
  label: 'Sala',
  value: _livingRoomLight,
  isAmbient: isAmbient,
  onChanged: (val) {
    setState(() { _livingRoomLight = val; });
  },
),
const SizedBox(height: 6),

// --- SECCIÓN 3: ACTUADOR LÁMPARA COCINA ---
_buildIotRow(
  icon: Icons.kitchen,
  label: 'Cocina',
  value: _kitchenLight,
  isAmbient: isAmbient,
  onChanged: (val) {
    setState(() { _kitchenLight = val; });
  },
),
const SizedBox(height: 8),

// --- SECCIÓN 4: TERMOSTATO (INTERACCIÓN SIN TECLADO) ---
if (!isAmbient) ...[
  const Text('Termostato', style: TextStyle(fontSize: 10,
color: Colors.grey)),
  Row(
    mainAxisAlignment: MainAxisAlignment.center,
    children: [
      IconButton(
        icon: const Icon(Icons.remove_circle_outline, color:
Colors.redAccent, size: 20),
        onPressed: () => setState(() => _targetTemperature-
-),
      ),
      Text(
        '$_targetTemperature°C',
        style: const TextStyle(fontSize: 15, fontWeight:
FontWeight.bold),
      ),
      IconButton(

```

```

        icon: const Icon(Icons.add_circle_outline, color:
Colors.greenAccent, size: 20),
        onPressed: () => setState(() =>
_targetTemperature++),
      ),
    ],
  ),
] else ...[
  // En modo ambiente solo mostramos el texto plano para
ahorrar batería
  Text('Termostato: $_targetTemperature°C', style: const
TextStyle(fontSize: 11, color: Colors.white60)),
],
),
),
),
);
},
);
},
);
}

// Componente reutilizable para construir los renglones de control IoT
Widget _buildIotRow({
  required IconData icon,
  required String label,
  required bool value,
  required bool isAmbient,
  required ValueChanged<bool> onChanged,
}) {
  return Container(
    padding: const EdgeInsets.symmetric(horizontal: 10, vertical: 4),
    decoration: BoxDecoration(
      color: isAmbient ? Colors.transparent : Colors.white10,
      borderRadius: BorderRadius.circular(10),
    ),
    child: Row(
      mainAxisAlignment: MainAxisAlignment.spaceBetween,
      children: [
        Row(
          children: [
            Icon(icon, size: 16, color: value && !isAmbient ? Colors.yellow :
Colors.grey),
            const SizedBox(width: 6),
            Text(label, style: const TextStyle(fontSize: 12)),
          ],
        ),
        // Si está en modo ambiente, pintamos un texto de estado estático en
lugar del switch
        isAmbient
          ? Text(value ? "ON" : "OFF", style: const TextStyle(fontSize: 11,
fontWeight: FontWeight.bold))

```

```
reloj : SizedBox(  
    height: 24,  
    scale: 0.8, // Encogemos el switch para que quepa bien en el  
  
    child: Switch(  
        value: value,  
        activeColor: Colors.cyanAccent,  
        onChanged: onChanged,  
    ),  
),  
],  
),  
);  
}  
}
```

Recuerda modificar tu archivo `Lib/app.dart` para que apunte a `home: const WatchIotScreen()`.

4. Guía para Probar en el Emulador

1. **Validación de Red:** Al iniciar la app, el emulador debe realizar la petición HTTP y pintar la temperatura actual real de la Ciudad de México de forma asíncrona. Si cambia a `...`, hay un bloqueo de red.
2. **Simulador de Modo Ambiente:** En la ventana de herramientas del emulador, ir a la pestaña **Wear OS**. Ahí verán un interruptor llamado **"Ambient Mode"**.
 - Al activarlo, verán cómo los componentes `Switch` interactivos y los botones de más/menos del termostato desaparecen instantáneamente del renderizado, transformándose en etiquetas de texto plano gris sobre fondo negro.
3. **Scroll e Interacción:** Al desactivar el modo ambiente, los alumnos usarán el cursor del mouse para arrastrar la pantalla emulando el dedo sobre el cristal del reloj, demostrando cómo opera el `SingleChildScrollView` dentro del contenedor recortado por la máscara del círculo físico.